

Python Notes – Comprehension Section

55 Examples with Code, Output, and Explanation (Compact Layout)

1. Squares 1–10

Code:

```
squares = [x**2 for x in range(1, 11)]
```

Output:

```
[1, 4, 9, ..., 100]
```

Explanation:

Squares of numbers 1 through 10

2. Even numbers 1–20

Code:

```
evens = [x for x in range(1,21) if x % 2 == 0]
```

Output:

```
[2, 4, 6, ..., 20]
```

Explanation:

Filter numbers divisible by 2

3. Uppercase words

Code:

```
upper = [w.upper() for w in words]
```

Output:

```
['APPLE', 'BANANA', 'CHERRY']
```

Explanation:

Convert each string to uppercase

4. Extract vowels

Code:

```
vowels = [ch for ch in text if ch in 'aeiou']
```

Output:

```
['o', 'e', 'e', 'i', 'o']
```

Explanation:

Collect vowels from a string

5. Squares of even numbers

Code:

```
squares = [x**2 for x in range(1,11) if x%2==0]
```

Output:

```
[4, 16, 36, 64, 100]
```

Explanation:

Square only even numbers

6. Replace negatives with 0

Code:

```
positive = [x if x>0 else 0 for x in nums]
```

Output:

```
[1,0,3,0,5]
```

Explanation:

Conditional expression inside comprehension

7. First letters of words

Code:

```
first_letters = [word[0] for word in sentence.split()]
```

Output:

```
['P','i','v']
```

Explanation:

Get first character of each word

8. Tuples (n, n²)

Code:

```
pairs = [(x, x**2) for x in range(1,6)]
```

Output:

```
[(1,1),(2,4),(3,9),(4,16),(5,25)]
```

Explanation:

Create tuple pairs

9. Flatten 2D list

Code:

```
flat = [num for row in matrix for num in row]
```

Output:

```
[1,2,3,4,5,6]
```

Explanation:

Flatten nested lists

10. Divisible by 2 and 3

Code:

```
nums = [x for x in range(1,31) if x%2==0 and x%3==0]
```

Output:

```
[6,12,18,24,30]
```

Explanation:

Number divisible by both

11. Unique chars from word (set)

Code:

```
chars = {ch for ch in 'banana'}
```

Output:

```
{'b', 'a', 'n'}
```

Explanation:

Set removes duplicates

12. Set of squares 1–5

Code:

```
squares = {x**2 for x in range(1,6)}
```

Output:

```
{1, 4, 9, 16, 25}
```

Explanation:

Set comprehension for unique squares

13. Unique vowels (case-insensitive)

Code:

```
vowels = {ch for ch in sentence.lower() if ch in 'aeiou'}
```

Output:

```
{'o', 'a', 'i'}
```

Explanation:

Collect unique vowels

14. Numbers < 10 (set)

Code:

```
small = {x for x in nums if x < 10}
```

Output:

```
{5, 8, 3}
```

Explanation:

Filter set elements

15. Odd numbers 1–20 (set)

Code:

```
odds = {x for x in range(1,21) if x%2!=0}
```

Output:

```
{1, 3, 5, ..., 19}
```

Explanation:

Generate odd numbers

16. Dict: number->square

Code:

```
squares = {x: x**2 for x in range(1,6)}
```

Output:

```
{1:1,2:4,3:9,4:16,5:25}
```

Explanation:

Map keys to computed values

17. Uppercase dict keys

Code:

```
upper = {k.upper(): v for k,v in fruits.items()}
```

Output:

```
{'APPLE':10, 'BANANA':20}
```

Explanation:

Transform dictionary keys

18. Swap keys and values

Code:

```
swapped = {v: k for k,v in d.items()}
```

Output:

```
{1:'a',2:'b',3:'c'}
```

Explanation:

Invert mapping

19. Filter dict by value>10

Code:

```
filtered = {k:v for k,v in prices.items() if v>10}
```

Output:

```
{'banana':15, 'cherry':25}
```

Explanation:

Keep items with value > 10

20. Number->cube dict

Code:

```
cubes = {x: x**3 for x in range(1,6)}
```

Output:

```
{1:1,2:8,3:27,4:64,5:125}
```

Explanation:

Compute cubes

21. Merge lists into dict

Code:

```
merged = {k:v for k,v in zip(keys, values)}
```

Output:

```
{'a':1, 'b':2, 'c':3}
```

Explanation:

Zip two lists into a dict

22. Word lengths

Code:

```
lengths = {w: len(w) for w in words}
```

Output:

```
{'python':6, 'is':2, 'awesome':7}
```

Explanation:

Map word to its length

23. Even numbers and halves

Code:

```
even_half = {x: x/2 for x in range(2,11) if x%2==0}
```

Output:

```
{2:1.0, 4:2.0, 6:3.0, 8:4.0, 10:5.0}
```

Explanation:

Compute half for evens

24. Matrix transpose (list)

Code:

```
transpose = [[row[i] for row in matrix] for i in range(3)]
```

Output:

```
[[1,4],[2,5],[3,6]]
```

Explanation:

Transpose by nested comprehensions

25. Multiplication table 1–5

Code:

```
table = [[i*j for j in range(1,6)] for i in range(1,6)]
```

Output:

```
[[1,2,...,5],...]
```

Explanation:

Nested comprehension for table

26. Cartesian product

Code:

```
cartesian = [(a,b) for a in A for b in B]
```

Output:

```
[(1,4),(1,5),(2,4),(2,5),(3,4),(3,5)]
```

Explanation:

All ordered pairs from two lists

27. Generator: cubes

Code:

```
cubes = (x**3 for x in range(1,6))
```

Output:

generator object

Explanation:

Lazy sequence producing cubes

28. Generator: evens

Code:

```
evens = (x for x in range(20) if x%2==0)
```

Output:

generator object

Explanation:

Generator filtering evens

29. Sum of squares (generator)

Code:

```
total = sum(x**2 for x in range(1,6))
```

Output:

55

Explanation:

Use generator inside sum()

30. Generator: words > 3

Code:

```
gen = (w for w in words if len(w)>3) for w in gen: print(w)
```

Output:

lion tiger

Explanation:

Iterate over filtered generator

31. Reverse each word

Code:

```
reversed_words = [word[::-1] for word in sentence.split()]
```

Output:

```
['nohtyP', 'si', 'lufrewop']
```

Explanation:

Reverse string slices per word

32. Perfect squares up to 100

Code:

```
perfect_squares = [x for x in range(1,101) if (x**0.5).is_integer()]
```

Output:

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Explanation:

Check integer square roots

33. Divisible by 3 but not 5

Code:

```
nums = [x for x in range(1,51) if x%3==0 and x%5!=0]
```

Output:

```
[3, 6, 9, 12, 18, ..., 48]
```

Explanation:

Combined condition

34. Pairs $x < y$

Code:

```
pairs = [(x,y) for x in range(1,6) for y in range(1,6) if x<y]
```

Output:

```
[(1, 2), (1, 3), ..., (4, 5)]
```

Explanation:

Ordered pairs where first < second

35. ASCII values lowercase

Code:

```
ascii_values = [ord(ch) for ch in 'abcdefghijklmnopqrstuvwxyz']
```

Output:

```
[97, 98, ..., 122]
```

Explanation:

ord() on characters

36. Words longer than 4 chars

Code:

```
long_words = [w for w in sentence.split() if len(w)>4]
```

Output:

```
['comprehension', 'makes', 'short']
```

Explanation:

Filter by length

37. Primes 2–50

Code:

```
primes = [x for x in range(2,51) if all(x%i!=0 for i in range(2,int(x**0.5)+1))]
```

Output:

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
```

Explanation:

Primality test with all()

38. Extract digits from string

Code:

```
digits = [int(ch) for ch in text if ch.isdigit()]
```

Output:

```
[1, 2, 3, 4, 5]
```

Explanation:

Filter digit characters and convert

39. Flatten 3D list

Code:

```
flat = [num for matrix in arr for row in matrix for num in row]
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

Explanation:

Triple nested flattening

40. Square odd, keep even

Code:

```
modified = [x**2 if x%2!=0 else x for x in nums]
```

Output:

```
[1, 2, 9, 4, 25]
```

Explanation:

Ternary expression inside comprehension

41. Unique words case-insensitive

Code:

```
unique_words = {w.lower() for w in sentence.split()}
```

Output:

```
{'python', 'is', 'fun'}
```

Explanation:

Case normalization then set

42. Unique consonants

Code:

```
unique_consonants = {ch for ch in text if ch not in 'aeiou'}
```

Output:


```
{'c','m','p','r','h','n','s','o','i','e'}
```

Explanation:

Filter out vowels

43. Odd squares 1–20 (set)

Code:

```
odd_squares = {x**2 for x in range(1,21) if x%2!=0}
```

Output:

```
{1, 9, 25, 49, 81, 121, 169, 225, 289, 361}
```

Explanation:

Squares of odd integers

44. Unique word lengths

Code:

```
word_lengths = {len(w) for w in sentence.split()}
```

Output:

```
{2, 4, 6, 7}
```

Explanation:

Set of distinct lengths

45. Char frequency dict

Code:

```
freq = {ch: text.count(ch) for ch in set(text)}
```

Output:

```
{'b':1, 'a':3, 'n':2}
```

Explanation:

Count occurrences using str.count()

46. Filter dict keep evens

Code:

```
filtered = {k:v for k,v in data.items() if v%2==0}
```

Output:

```
{'b':4, 'c':8}
```

Explanation:

Dictionary filtering by value parity

47. Celsius to Fahrenheit dict

Code:

```
fahrenheit = {city: (temp*9/5)+32 for city,temp in celsius.items()}
```

Output:

```
{'Delhi':86.0, 'Mumbai':89.6, 'Shimla':68.0}
```

Explanation:

Apply formula to values

48. Word->length if >3

Code:

```
word_len = {w: len(w) for w in words if len(w)>3}
```

Output:

```
{'elephant':8, 'tiger':5}
```

Explanation:

Conditional inclusion in dict

49. Number: Even/Odd classification

Code:

```
num_type = {n: ('Even' if n%2==0 else 'Odd') for n in range(1,11)}
```

Output:

```
{1: 'Odd', 2: 'Even', ...}
```

Explanation:

Ternary used for values

50. Merge skip None

Code:

```
clean = {k:v for k,v in zip(keys,values) if v is not None}
```

Output:

```
{'a':1, 'c':3, 'd':4}
```

Explanation:

Skip None values while zipping

51. 3x3 Matrix creation

Code:

```
matrix = [[j for j in range(i,i+3)] for i in range(1,10,3)]
```

Output:

```
[[1,2,3], [4,5,6], [7,8,9]]
```

Explanation:

Generate rows programmatically

52. Transpose 3x3

Code:

```
transpose = [[row[i] for row in matrix] for i in range(3)]
```

Output:

```
[[1,4,7], [2,5,8], [3,6,9]]
```

Explanation:

Classic transpose

53. Pairs with even sum

Code:

```
pairs = [(i,j) for i in range(1,6) for j in range(1,6) if (i+j)%2==0]
```

Output:

```
[(1,1),(1,3),(1,5)...]
```

Explanation:

Filter pairs by sum parity

54. 2-letter combinations

Code:

```
combinations = [a+b for a in s for b in s if a!=b]
```

Output:

```
['AB','AC','BA','BC','CA','CB']
```

Explanation:

All orderings without repetition of same letter

55. Palindromes dict

Code:

```
palindromes = {w: w[::-1] for w in words if w==w[::-1]}
```

Output:

```
{'level': 'level', 'madam': 'madam'}
```

Explanation:

Include only palindromic words in dict